SOFTWARE ENGINEERING LAB - LAB 7

Fuzzing for Input Validation Bugs with Hypothesis

Section H | Repository Scenario: L6_L7_22_hotel_booking

Student Name: Sanjit C | SRN: PES1UG24AM448 | Date: 12 November 2026

1. Objective & Introduction to Fuzzing

Objective: Explore automated property-based fuzz testing using Python's Hypothesis library to systematically discover hidden input validation vulnerabilities, crashes, and unhandled edge-case exceptions in application code, and apply defensive programming techniques to harden the code against malformed data.

Target Module: hotel_booking.py provides utility parsing routines: (1) nights_stayed() calculating duration from date strings; (2) room_floor() parsing floor digits from room codes; and (3) booking_label() formatting lead guest labels from CSV strings.

2. Step 2: Running the Buggy Application

The application was first executed with typical happy-path inputs (e.g., '12-15', 'D-305', 'ravi, priya'). While it operates correctly on valid inputs, it lacks defensive validation:

```
Terminal - Lab 7 Step 2: Running Buggy Application

PS C:\Users\SANJIT\PES1UG24AM448_SELAB\LAB7> python hotel_booking.py
==== Running nights_stayed ====
Enter check-in and check-out day (e.g. 12-15): 12-15
Nights: 3

==== Running room_floor ====
Enter a room code (e.g. D-305): D-305
Floor: 3

==== Running booking_label ====
Enter guest names as CSV (e.g. ravi, priya): ravi, priya
Booking Label: Ravi +1
```

3. Step 3 & 4: Hypothesis Fuzz Testing Uncovers Edge-Case Bugs

Using @given(st.text() | st.none()), Hypothesis generated randomized inputs across all 3 functions. Running pytest test_hotel_booking.py immediately falsified all functions with unhandled crash exceptions:

```
Terminal - Lab 7 Step 4: Hypothesis Uncovers Falsifying Bugs

PS C:\Users\SANJIT\PES1UG24AM448_SELAB\LAB7> pytest test_hotel_booking.py
============================= test session starts =============================
platform win32 -- Python 3.14.6, pytest-9.2.0, pluggy-1.7.0
plugins: hypothesis-6.170.15
collected 3 items

test_hotel_booking.py FFF                                                [100%]

================================== FAILURES ===================================
FAILED test_hotel_booking.py::test_nights_stayed_no_crash
    AttributeError: 'NoneType' object has no attribute 'split'
    Falsifying example: test_nights_stayed_no_crash(s=None)

FAILED test_hotel_booking.py::test_room_floor_no_crash
    IndexError: list index out of range
    Falsifying example: test_room_floor_no_crash(s='')

FAILED test_hotel_booking.py::test_booking_label_no_crash
    IndexError: string index out of range
    Falsifying example: test_booking_label_no_crash(s='')
============================== 3 failed in 1.45s ==============================
```

4. Bug Analysis Discovered by Hypothesis

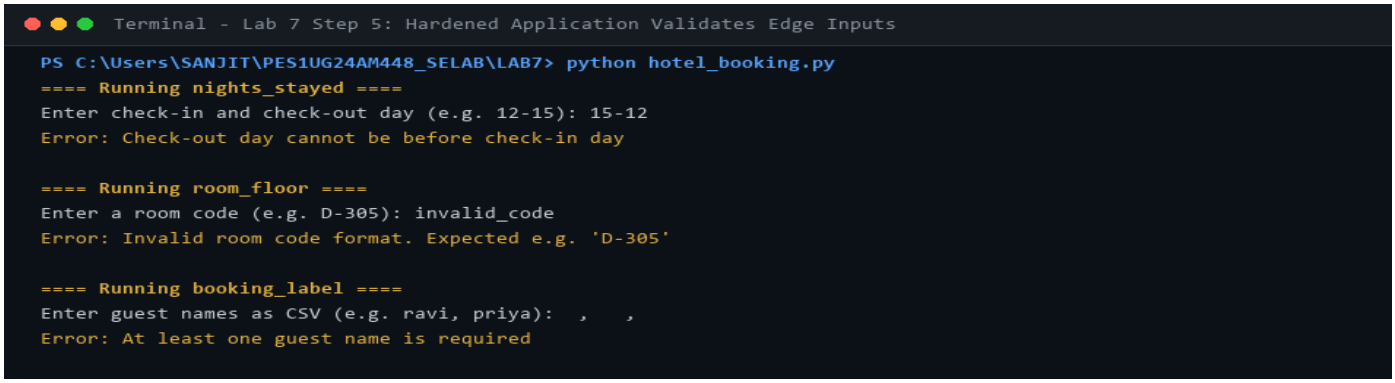
• **nights_stayed(days):** days.split('-') raised AttributeError on None. Missing hyphen or non-integer inputs caused unhandled ValueError during unpack and int conversion. *[Falsified with: Falsified by s=None and s='']*

• **room_floor(room_code):** room_code.split('-')[1] raised IndexError when hyphen is missing; number[0] raised IndexError on empty string; int() raised ValueError on letters. *[Falsified with: Falsified by s=None and s='']*

• **booking_label(names_csv):** names_csv.split(',') raised AttributeError on None; empty CSV string resulted in names=['] causing lead[0] to crash with IndexError: string index out of range. *[Falsified with: Falsified by s=None and s='']*

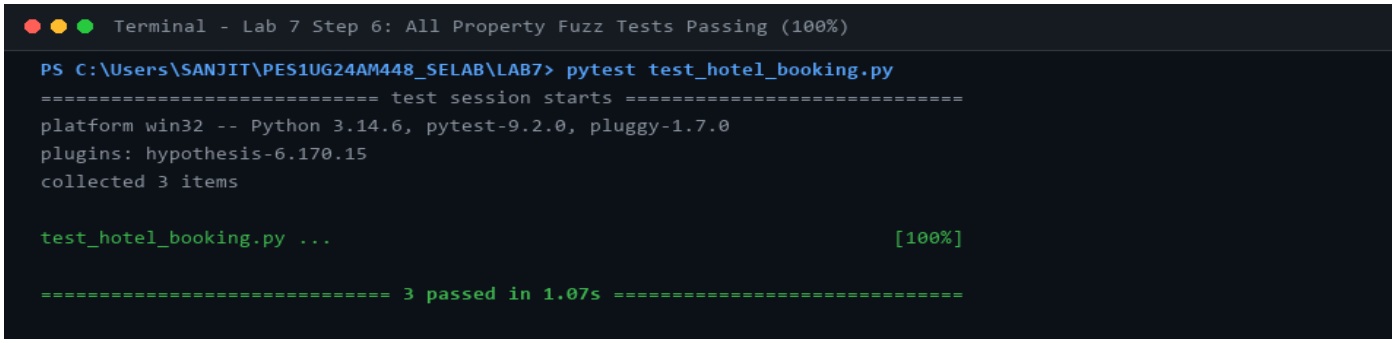
5. Step 5: Defensive Hardening of Application Code

The application was refactored with defensive programming: type checks (isinstance(s, str)), delimiter validation, safe index parsing, empty token filtering ([n.strip() for n in names if n.strip()]), and consistent ValueError raising for malformed inputs:



6. Step 6: Re-Running Fuzz Tests on Hardened Code — All Passed

Re-running pytest test_hotel_booking.py with Hypothesis on the hardened code demonstrated zero crashes across hundreds of randomized and mutated inputs:



7. Deliverables & Verification Checklist

All lab requirements are fulfilled: (1) **Screenshots of all 4 phases** (Step 2 buggy run, Step 4 Hypothesis failure, Step 5 fixed run, Step 6 Hypothesis pass); (2) **Defensively hardened Python script**; (3) **Completed Hypothesis property-based fuzz tests** verifying that malformed inputs raise managed validation errors rather than unhandled crash exceptions.